COMP2040: Operating Systems

Checkpoint 1: Progress Report

Group 10: Nguyen Vu Nhat Minh (V202401314) Do Chi Hai (V202401459)
College of Engineering and Computer Science, VinUniversity

All four Checkpoint 1 schedulers and resource deadlock detection are implemented in `ossim`. Design choices are recorded in `DECISION.md` (D-4, D-7 to D-10) and every hand calculation in `docs/checks/`.

1 Data Structures and Design

Every scheduler fills the `scheduler_t` hooks and keeps its READY processes in `queue_t`, an array of PCB pointers (at most 64). New and preempted processes join the tail. `q_pop_min` removes the smallest key and keeps queue order on ties, so ties are FIFO, not by pid.

(a) SJF. One queue, key `proc_burst_remaining`: the *current* CPU burst, not all future work. Never preempts; a process whose next operation is `cpu` keeps the CPU without a new dispatch.

(b) SRTF. Same queue and key. `should_preempt` is true when `q_min_key` is *strictly* smaller than the runner's remaining burst, so equal bursts do not swap every tick.

(c) PRIO. Key `priority` (smaller is higher). A strictly higher priority preempts at once. The quantum counter `quantum_used` grows in `on_tick` and is reset at each dispatch; on expiry the runner goes to the tail only if an equal priority is ready, so equal priorities share the CPU round-robin and lower ones cannot force a rotation.

(d) MLFQ. Three queues, one per level; level ℓ has quantum $q \cdot 2^{\ell}$ and `pick` takes the head of the highest non-empty level. `on_tick` demotes a process as soon as its quantum is used up (the bottom level only restarts it). After expiry it keeps the CPU only if nobody is ready at its new level or higher; a process ready at a higher level preempts at once and the preempted one keeps its level. No hook runs every tick, but each tick calls `should_preempt` or `pick`, so both start with the boost: at $t = 20, 40, \ldots$, once per tick, levels 1 and 2 move in order to the tail of level 0 and every live process, blocked or running, gets level 0.

(e) Deadlock detection. The algorithm of Sec. 8.6.2 with `Available`, `allocated[pid]` and `requested[pid]` from each `resource_t`: `Work` starts as Available, a process holding nothing is finished, and any process whose Request fits in `Work` is assumed to finish and return its Allocation. Those left are deadlocked and printed once, in pid order, as `DEADLOCK t=<tick> pids: …`; a static array of the last reported set prevents repeats. It handles several instances per resource. A process blocked on a semaphore has no Request and so counts as able to finish. There is no recovery.

Cost and prediction. Selection and preemption tests scan the queue in $O(n)$; detection is $O(n^2 m)$ for n processes and m resources. Both are small at 64 processes. `ossim` knows each burst length; a real OS would predict it with exponential averaging, $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.

2 How We Checked Them

Each case was traced by hand from the rules in the Guide before running `ossim` (`--no-paging`; quantum 2 for PRIO and MLFQ). The teammate who did not write the code redid part of each trace in review.

Scheduler, `arrivals.txt`	Average in ticks (hand = simulator)			
	Waiting	Response	Turnaround	Switches
SJF	10.00	9.00	17.83	8
SRTF	9.83	9.00	17.67	9
PRIO	12.83	8.33	20.67	12
MLFQ	15.33	0.00	23.17	19

- **Edge cases.** `sched_edges.txt` (FIFO ties, same-tick events, back-to-back bursts) for SJF, SRTF and PRIO; `prio_check.txt` for the fresh quantum after preemption; `mlfq_check.txt` for the bottom level and the boost, which alone gives C a response time of 1 (waiting 1.67, response 0.33, turnaround 13.00, 5 switches). SJF gives the textbook waiting time 3.00 on bursts 24, 3, 3.
- **Deadlock.** On `deadlock.txt` (RR, $q = 2$, check every 5 ticks) B blocks at the end of tick 11 and A at the end of tick 12, so the first report is `DEADLOCK t=15 pids: 1 2`, printed once; both of us derived this. `deadlock_multi.txt` (one resource, two instances) must *not* report at $t = 4$, since B can still finish and free an instance, and must report at $t = 6$. `no_deadlock_multi.txt` is blocked but never deadlocked.
- **Integration.** `producer_consumer.txt` finishes under all six schedulers, and `mixed.txt` with paging, including one frame, under the four new ones.

Every value above is a check in `tests/run.sh`; `check` and `check_no` also fail if `ossim` exits with an error. With GCC and `-std=c99 -Wall -Wextra -Werror`, `make test` gives **65 passed, 4 skipped (Checkpoint 2), 0 failed**.

3 Who Did What

Nguyen Vu Nhat Minh (Person A, `garan1610`): SJF, SRTF and PRIO with their hand checks, edge-case workloads and tests; stricter `check`/`check_no`; LF line endings for WSL; review of deadlock detection and MLFQ.

Do Chi Hai (Person B, `TheNameisSea`): MLFQ and deadlock detection with their hand checks, workloads and tests; team guide, git workflow and decision log; review of SJF, SRTF and PRIO.

Every change went through a pull request reviewed by the other member. AI assistants helped with implementation, hand checks and debugging; each of us can explain every line we submitted.

Limitations. PRIO can starve low priorities (no aging); detection ignores semaphores; only GCC was tested, as Clang was not available.